Angular Project Point

Date: / / Page no:

* We have Pipes concept in Angular and pipe() function in RxJS.

1. Pipes in Angular: A pipe takes in data as input and transforms it to the desired output

2) Pipe() function in RxJS: you can use pipes to link operators together. Pipes let you combine multiple functions into a single function.

The Pipe() function takes as its arguments the functions you want to combine, and returns a new function that, when executed, runs the composed function in sequence.

---

* An Observable is an entity that emits (or publishes) multiple data values (stream of data) over time and asynchronously

* Observers are also called listeners (or consumers) as they can listen or subscribe to get the observed data

* Subscriptions are the objects that are returned when you subscribe to an Observable.

- A Sublect is a special type of Observable that observers can also subscribe to it to receive published values but with one difference: The values are multicased to many observers.
- By default an RxJS Observable is unicast
- Unicast simply means that each subscribed observer has an independent execution of the observable while multicast means that the observable execution is shared by multiple observers

Note: Subjects are similar to Angular Event Emitters

So when using Subjects instead of plain Observables, all subscribed Observers will get the same values of emitted data.

Note:- Subject are also Observers ie they can also subscribe to other Observables and listen to published data.

Hot and Cold Observables.

Subjects are called Hot.

* RxJS provides, two types of Subjects:
  Behavior Subject and ReplaySubject.

---

* constructor (public messageService: MessageService) {}

  The messageService property must be public because you're going to bind to it in the template.

  Note: Angular only binds to public component properties.

Angular.

Dart can be used by developers along with typescript in version 2.0

Date: / / Page no:

version 1.0 vs. version 2.0

- The architecture of Angular JS is based on MVC whereas the architecture of Angular 2 is based on Service / Controller. (follow OOPS)
- Angular 2 is completely rewritten from the ground up and as a result the way we write applications with Angular JS and Angular 2 is very different. (Angular Javascript. typescript.)
- Angular 2 is 5 times faster compared to Angular JS.
- AngularJs was not built for mobile devices, whereas Angular 2 is designed with mobile support in mind.
- We can use Typescript, Dart, Purescript, Elm, etc

version 2.0 vs version 4.0

- It will simply be a change in some core libraries.
- Angular v4.0 is compatible with newer versions Typescript 2.1 and Typescript 2.2. This helps with better type checking and also enhanced IDE features for visual studio code.
- Angular 4 is simply, the next version of Angular 2.
- Upgrade of the version from 2.0 to 4.0 has reduced its bundled file size by 60%. The code generated is reduced and has accelarated the application development. Here the developed code can be used for prod mode and debug.
- Why skipped Angular 3.

Ans. @angular/core v.2.3.0
@angular/compiler v2.3.0
@angular/compiler-cli v2.3.0
@angular/http v2.3.0
@angular/router v.3.3.0. ←

Angular JS is a library → lib को किसी technology से integrate application develop किया जाता है

Angular 2 is a framework → उसमे whole application को developed किया जा सकता है

Node.JS : Node.JS is an open-Source, cross-Platform Javascript run time enviroment.

NPM :- It is a node.JS package manager for Javascript programming language. It is automatically installed with node.JS.
↓ node package manager

Angular CLI (command line interface) : is a tool to that allow us to create a project, build and run it on development server directly using command line.

Command: npm install -g @angular/cli
(g → global)

What is angular?

1. The angular is the newest form of AngularJS developed by Google, which is an open-source front-end development platform used for building mobile and desktop web application. Angular is rewritten by the same team that built AngularJS

2. Angular is one of the most popular frameworks for building client apps with HTML, CSS and Typescript.

Angular 9 (released on 7 feb 2020)

- ivy has arrived.
- Developed by Google
- front end / client side Javascript framework

Date: / / Page no:

Features.

- Speed and performance.
- Smaller application.
- Modular application.
- Cross - Platform (web, Mobile and Desktop)
- Single Page application (SPA)
- Result API
- use typescript

test.ts and karma.config.js (framework jasmine) → for test work

* Angular version numbers have three parts
1. Major 2. minor 3. Patch

(Breaking change) mayor version → developer are required
developer are require

eg. 5.2.9

mayor version ↓ ; patch version (Bugfix) ← 9 ; minor version (New feature) ↑ 2 ; (Reversion) ; Patch ↓ releases are low risk, bug fix releases. No developers assistance is required during update

---

- editorconfig → team work (one set all the rule here).
- gitignore = उन file को लिखना है जिसको git मे लिखना नहीं है
- polyfill.ts = browser के Related सारा configuration इसमे हि होता है।

जैसे हि कोई पहले हमारे URL को hit करता है तब हमारा compiler सबसे पहले main.ts के पास जाता है। और यहाँ पर मिलता है bootstrap module यानि कौन सा module पहले load करना है। तो यहाँ पर AppModule मिल गया तो अब वहां से app.module मे जायेगा (import को देखने पर) सबसे पहले. तो यहाँ पर मिलेगा कि सबसे पहले load कौन होगा. तब यहाँ पर हमे bootstrap मे AppComponent लिखा हुआ है यानि सबसे पहले AppComponent load होगा. finally ये Component load होगा तो यह Component load कहां पर होगा जहां पर इस component का selector होगा. ("APP component.ts का") selector हर एक component का instance होता है तो इसको आप जहाँ पर रख देंगे वहाँ पर हमारा component load हो जाता है, . यह selector हमारा bydefault index.html पर रखा मिल जाता है. यानि finally आपके browser मे index.html load होगा और वहीं पर ये app.component.ts file load हो जायेगा. आपके browser मे हमेशा index.html हि होता है और

Karma config.js — के अन्दर Jasmine एक framework होता है जो testing purpose से काम आता है।

वहि पर आके components load ~~होते है~~ // ~~इसलिए~~ हम इसको SPA बोलते है क्योकि यहा पर हमारा page एक हि है और यहि पर सारे component add होते रहते है। यानि ts file हमलोग logic लिखते है तो html file हमारे browser पर load होगा।

अगर आप ये सोचते है कि हम अलग से page नहीं लेगें हम अपना page यहि पर load करेगें तो सबसे पहले हम metadata templateUrl को हटा देगें और उसके जगह पर

template: '<h1> Hi, how are you ? </h1>'

(→ title सिर्फ)

अगर हम color के लिए सोचे तो हम अपना inline कर सकते है

styles: ['h1{color: green}']

Package.json and Package-lock.json.

Package.json — हमे ये बताता है कि हमे project बनाने मे कितने package कि जरूरत पड़ेगी

Package-lock.json → हमे ये बताता है कि कौन-कौन से package का use किया गया है project बनाने के लिए

यदि हम

"@angular/cli" : "9.0.2", जगह पर "8.0.2" लिखे और npm install run करे तो यह package-lock.json मे 8.0.2 install कर देगा

(करने पर इसको folder node_modules मे download हो जाता है)

(~) tild मे patch को check करके install करता है (→ upper version को check)

(^) carret मे minor और patch को check करके install करता है अगर कुछ नही है तो exact version मिलना चाहिए नही तो error देगा।

(by npm)

Modules:

logical grouping of component and Services.

default module : app.module.ts.

* every application needs at least one module that is known root module for angular application.

declaration मे component, directives, pipe — compile time मे show करता है और लिखना जरूरी है

imports मे module

providers के अन्दर service रखते है। ← इसमे यदि error आता है तो हमे नहि दिखता है पर Result नही आता है

For create module

ng g module company

for creating component inside company module

Command: ng g c company/employee-login

Decorators.

- Decorators are feature of TypeScript and are implemented as functions. The name of the decorator starts with @ symbol following by brackets and arguments. Since decorators are just function in TypeScript.
- Decorators are simply functions that return functions. These functions supply metadata to Angular about a particular class, property, value, or method.
- Decorators are invoked at runtime.
- Decorators allow you to execute functions. for example @Component executes the component function imported from angular 7

export class करने का मतलब उसको हम public कर रहे हैं ताकि कहि भी use कर सकते है

(component define करते हैं → Date; module define करते हैं → Page no.)

Types of Decorators

- Class decorators. eg @Component and @NgModule
- property decorators : for properties inside classes
  eg @input and @output.
- Method decorators. : for methods inside classes.
  eg @Host Listener → host के Response को listen करते हैं।
- Parameter decorators : for parameters inside class constructors.
  eg @inject.

- Each decorator has a unique role.
- सबको core library से import करना पड़ता है।

Demo

app.component.ts.

```
@HostListener('click', ['$event'])
onhostclick(event: Event){
    alert("hello"); }
```

Component

* Component is the basic building block of Angular, which means that an Angular application is a collection of components and one component is responsible for handling one view or part of the view.
* a component encapsulates the data, the HTML markup and the logic for a view.
* Component is nothing but a simple typescript class which contains following things
  * typescript class.
  * HTML template | Template URL
  * @Component decorator with metadata.

* Every angular application (by default APP component) has at least one component that is used to display the data on the view.

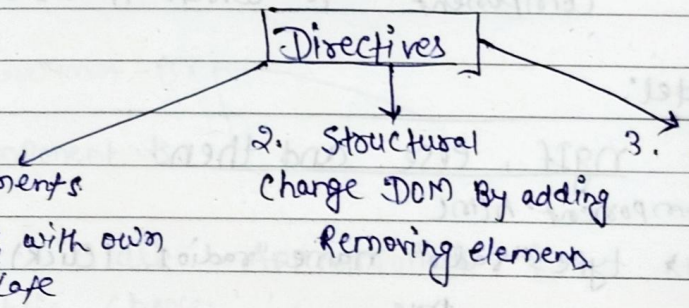
* Angular is a component-based architecture which allows us to work on smaller and more maintainable pieces that can also be used in different places

app.component.ts

```
@Component({
  ==
  preserveWhitespaces: true
})
```

Directives.

* Directives are elements which change the appearance or behavior of the Dom (Document object model) element. There are 3 types of the Directives mainly



2. Structural Directives

* Structural directives are responsible for the HTML layout. They shape or reshape the HTML view by simply adding or removing the elements in the Dom. These directives are the way to handle how the component or the element renders in a template.

Ng switch is actually a set of three, cooperating directives: Ng switch, Ng switch Case and Ng switch Default

Date: / / Page no:

* There are basically 3 structural directives available in Angular

* NgIf (*ngif) → true: added, false: remove
* NgFor (*ngfor)
* NgSwitch (*ngswitch)

→ It is a repeater directive - a way to represent a list of items.

3. Attribute Directives

* Attribute directive is a way to modify the appearance of the DOM element or component. There are 2 built-in Attribute Directives in Angular

* NgStyle: Angular provides a built-in NgStyle attribute to modify the element appearance and behavior.

* ngClass: This attribute is used to change the class attribute of the element in DOM or the component to which it has been attached.

* NgModel:

ngIf, else and then.

ng-template as such

app.component.html

```
<input type="radio" name="radio1" (click)="changevalue(true)">
                                    true
<input type="radio" name="radio1" (click)="changevalue(false)">false
<div *ngif="isvalid; then thenblock; else elseblock"></div>
<ng-template #thenblock>
  <div> this is valid </div>  </ng-template> </div>
<ng-template #elseblock> <div>
    This is not valid data using else block </div>
</ng-template>
```

Date: / / Page no:

App.Component.ts

```
export class AppComponent {
    isValid: boolean = true;
    changeValue(valid) {
        this.isValid = valid;
    }
}
```

---

ng-switch

App.component.html

```
<select (change)="setValue($event)">
    <option value="">- select -- </option>
    <option value="one"> one </option>
    :
    :
</select>
<div [ngSwitch]="choose">
<h3 *ngSwitchCase="one"> First </h3>
    :
    :
</div>
```

App.component.ts

```
export class AppComponent {
    public choose='';
    setValue(drp: any)
    { this.choose = drp.target.value; }
}
```

Bind to [ngSwitch]. You'll get an error if you try to set *ngSwitch because NgSwitch is an attribute directive, not a structural directive. Rather than touching the DOM directly, it changes the behavior of its companion directives. Bind to *ngSwitchCase and *ngSwitchDefault. The NgSwitchCase and NgSwitchDefault directives are structural directives ~~Rather than touching the~~ bcoz they add or remove elements from the DOM

ngfor → index of the current element in the ngfor loop by using index variable

Date: / / Page no:

App.component.html

even, odd is also there

```
<ul>
<li  *ngfor="let student of students; let i = index;
      let f = first; let l = last">
  {{i+1}} - {{student.name}} - {{f}} - {{l}}
</li>
</ul>
```

App.component.ts

```
export class AppComponent {
    student: any[] = [
    {
        'name' : 'Rahul kumar'
    },
    {
    3
        :
    {
    }

}
```

App.component.html

trackBy: trackbystudentid

```
<table border="1">
    <tr *ngfor="let s of students; let i = index">
    <td> {{i+1}} </td>
    <td> {{s.studentid}} </td>
    <td> {{s.name}} </td>
</tr> </table> <br> <button (click)="getmore
Student()">Get more students </button>
```

* ngIf = "condition"

[ngIf] = "condition"

Date: / / Page no:

*ngFor = "let t of todos; let i=index"
de-sugars it into
template = "ngFor: let t of todos; let i=index"
which de-sugars into
<template ngFor [ngForOf]="todos" > </template>

App. Component. ts

```
export class AppComponent {
    student: any[];
    Constructor() {
      this.students = [
        { studentid: 1  name: 'chandan' };
      ];
    }
    getmorestudents(): void {
    }
    trackbystudentid(index: number, student: any): string {
        return student.studentid
    }
```

Grouping in ngfor

app. Component. ts

```
export class AppComponent {
    constructor() {
    }
    countrydetails: any[] = [
      {
        'country': 'India',
        'people': [ { 'name': "Ajeet Kumar" }, ..... ];
        country = "UK"
```

ng means Angular

Date: / / Page no:

App.component.html

```
<ul  *ngfor = "let group of countrydetails">
   <li> {{ group.country }} </li>
    <ul>
       <li  *ngfor = "let person of group.people">
          {{ person.name }}
       </li> </ul> </ul>
```

---

ngStyle → Em font fir change कर सकते है??

- The ngStyle directives lets you set a given Dom elements Style properties.

```
<div [ngstyle] = "{ background-Color' : green }" </div>.
```

- ngstyle becomes much more useful when the value is dynamic.

```
<div [ngstyle] = "{ background-Color' : person.country ==
      'UK' ? 'green' : 'red' }" > </div>
```

App.component.ts

```
export   Class   AppComponent {
    Constructor() {
    }
    people : any[] = [
     {
        "name" : "Adeet Kumar",
        "Country" : "India"
     },
     ];
}
```

getColor(country) {
switch(country) {
case 'India' : return 'green'
case 'UK' : return 'blue'
}
}

the let keyword before person creates a template input variable called person.

App.Component.html

Date: / / Page no:

```
<ul *ngFor = "let person of people" >
  <li [ngstyle] = "{ 'color' : getColor (person.country)}" >
    {{ person.name }} - ({{ person.country }})

  </li> </ul>
```

---

ngClass

ngclass directives allows us to set the CSS class dynamically for a Dom element.

- ngClass with String
- ngClass with array
- ngclass with object
- ngclass with Component method

ex.

App.component.ts

```
constructor() {}
users = [
  'Chandan', 'Ajeet' ];

getCSSClass(flag:String) {
  let cssclass;
  if(flag == 'mode') {
    cssClass = {
      'one' : true;
      'two' : true }
  return cssclass
  } else { cssclass = {'one': false
    'two' : true } }
```

App.component.css.

```
.one {
  color: blue; }
.two { font-size: 30 px ; }
.three { color: green; }
.four { font-size : 15 px; }
```

App.component.html

as a array "['one', 'two']"

```
<p [ngClass] = " 'one two' " > using ngClass with String example </p>
<div *ngFor = "let user of users" [ngclass] = " 'one four' ">
  {{ user }}
</div> <p [ngclass] = "{'one' : true, 'three' : false}" > Example using ngClass as object </p> <div [ngclass] = "getCSSClass('mode')" > Example using ngclass with Component method </div>
```

Angular Material.

Date: / / Page no:

Angular material is a UI component library for Angular developers. Angular Materials reusable UI components help in constructive attractive, consistent and functional web pages and web applications while adhering to modern web design principles like browsers Portability, device independence and graceful degradation.

website material.angular.io

- It supports shadows and bold colors
- The colors and shades remain uniform across various platforms and devices.
- Angular material classes are created in such a way that the website can fit any screen size.
- The websites created using Angular Material are fully compatible with PC, tablets and mobile devices.

install.

initial 3 main dependencies

npm install --save @angular/material @angular/cdk @angular/animations.

जो कि AppComponent में use करना होता.

उसके बाद use करने के लिए import और export करना होगा AppModule.ts

---

Reactive form

There are primarily two types of forms available in Angular

1. template-driven forms 2. Reactive forms.

Router → Library ☺

Date: / / Page no:

Angular router is an official Angular routing library written and maintain by the Angular core team.

- It activates all required Angular components to compose a page when a user navigates to a certain URL.
- it lets users navigate from one page to another without page reload.
- It updates the browser's history so the user can use the back and forward buttons when navigating back and forth between pages

Angular Router allows us to

- redirect a URL to another URL.
- resolve data before a page is displayed.
- run scripts when a page is activated or deactivated
- lazy load parts of our application.

How the Angular Router Work.

Angular Router performs the following 7 steps inorder

Step 1 - Parse the URL

- In Step 1 of the Routing process, Angular router takes the browser URL and parses it as the URL tree.
- A URL tree is a data structure that will later help Angular Router identify the router state tree in step 3.

To Parse the URL, Angular uses the following conventions

Date: / / Page no:

- / - Slashes divide URL segments.
- () - Parentheses specify secondary routes
- : - a colon specifies a named router outlet
- ; - a semicolon specifies a matrix parameters
- ? - a question mark separates the query string parameters.
- # - a hashtag specifies the fragment.
- // - a double slash separates multiple secondary routes

Step 2 - Redirect

- Before Angular router uses the URL tree to create a router state, it checks to see if any redirects should be applied. There are 2 kinds of redirect

- local redirect: when redirectTo does not start with a slash. replaces the entire URL segment

  Example: { path: 'one', redirectTo: 'two' }

- absolute redirect: when redirectTo starts with a slash. replaces the entire URL.

  - Example { path: 'one', redirectTo: '/two' }

- Only one redirect is applied.

Step 3 - identify the router state.

- Angular router traverses the URL tree and matches the URL segments against the path configured in the router configuration.

- If a URL segment matches the path of a route, the route's child routes are matched against the remaining URL segments until all URL segments are matched.
- If no complete match is found, the router backtracks to find a match in the next sibling route.

Step 4 - Guard - run guards

- At the moment, any user can navigate anywhere in the application anytime. that's not always the right thing to do.
- perhaps the user is not authorized to navigate to the target component
- maybe the user must login (authenticate) first.
- maybe you should fetch some data before you display the target component.
- you might want to save pending changes before leaving a component
- you might ask the user if its ok to discard pending changes rather than save them.
- you can add guards to the route configuration to handle these scenarios.

Step 5 - Resolve - run Resolve
it resolves the required data for the router state.

Step 6 - Activate
it activates the Angular components to display the page.

Step 7 - Manage.
- It manages navigation and repeats the process when a new URL is listened from the user.

Date: / / Page no:

Router Outlet.

- Router Outlet is a dynamic component that the router uses to displays views based on router navigations.
- Router Outlet is a Routing component. An Angular Component with a RouterOutlet that displays views based on router navigations.
- The role of <router-outlet> is to mark where the router displays a view. (This is the location where the Angular will insert the component we want to route to on the view).
- The <router-outlet> tells the router where to display routed views
- The RouterOutlet is one of the router directives that became available to the AppComponent bcoz AppModule imports APPRoutingModule which exported RouterModule

Wildcard Route.

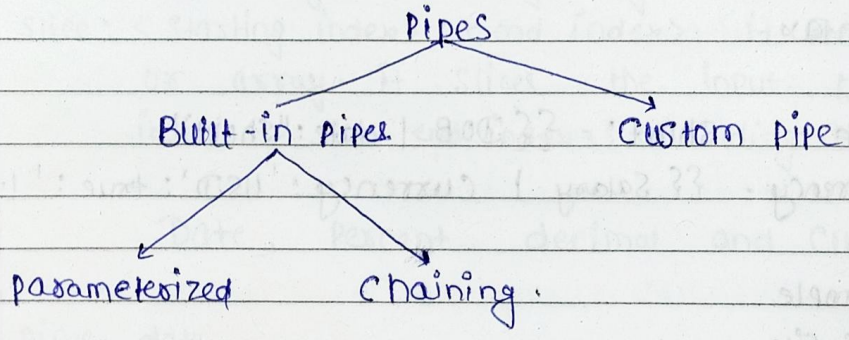
* if you add a wildcard route as the first route, no other routes would be reached and the wildcard route would always be matched. As a result, you should always add a wildcard route as the last route in your router configuration

Pipes in angular.

Date: / / Page no:

- Angular pipes can be used to transform data to desired output.
- Pipes takes in a data input and transform data to a different output.
- Using pipe operator (|), we can apply the pipe's feature to any of the property in our Angular project.
- Pipes (|) in Angular are used to transform the data before displaying it in a browser. Angular provides a lot of built-in pipes to translate the data before displaying it into the browser and as we know, Angular lets us extend its feature, we can even create custom pipes in Angular.

Types of Pipes.



Example

ts file

```
export class AppComponent {
    employees : any[] = [
      {
        code: 'emp001', name: 'chandan', salary: 85000, dob: 'dd/mm/yy
      },
    ];
```

App. component.html.

```
<table>
  <tr> <th> Emp code </th> <th> Emp Name </th>
  <th> Salary </th> <th> Date of Birth </th> </tr>
  <tr *ngFor = "let employee of employees">
  <td>{{employee.code}} </td>.
  <td> {{employee.name | uppercase}} </td>
  <td> {{ employee.Salary }} </td>
  <td> {{ employee.dob}} </td> </tr> </table>
```

PARAMETERIZED PIPES.

- We can pass any number of parameters to the pipe using a colon (:)
- data | pipeName : Parameter 1 : Parameter 2 : ...... : Parameter n.
  Syntax

Date - Short :- {{ DOB | date:"Short"}}
Currency - {{ Salary | Currency : 'USD' : true : '1.3-3'}}

example

ts file

```
Salary : number = 65000;
dob = new Date( 1986, 8, 2);
```

html.

```
<div> {{ dob}} <br>
medium : {{ dob | date: "medium"}} </br>
specific formate: {{ dob | date : "dd/MM/yyyy" }}.
medium time : {{ dob | date : "mediumTime" }}
Salary :- {{ Salary }}
Salary :- {{ Salary | currency : "USD" : true }}
</div>
```

lowercase

Short time
second [illegible]
full time
[illegible]

Fulldate :- {{ dob | date: "fulldate" | uppercase}}

Multiple Pipes (Chaining Pipes).

Date: / / Page no:

- We can use multiple pipes with the same data at the same. This also reffered as Chaining Pipes.
- data | Pipe 1 : Parameter 1 : Parameter 2 : Parameter 3 .... : Parameter |Pipe 2 : Parameter 1 : ---- : Parameter n | Pipe 3 : Parameter 1 ; Parameter 2 : ---- : Parameter n | .. Pipe n : Parameter 1 .... : Para n.

Pipes - Uppercase, lowercase, titlecase, Slice.

- Uppercase Converts the String to upper case.
- Lowercase , · Titlecase
- Slice : <Starting index> it works on String fields or Array. it slices the input from the index till the end of the String or array length.
- slice : <Starting index> : <end index> it work on String fields or array. it slices the input from the starting index to the end index. (excluding the end index)

Date, Percent, decimal and Currency pipe.

Pipes - date

- There are many data type pipes available in Angular Following are some of the important ones
- ShortDate → 7/25/2020 · It Converts the date to a Short date.
- longDate : It provides the long date for the date. provides full month name then day and year. July 25, 2020
- FullDate : It provides the full date for the date i.e. date, month date, year → Saturday, July 25, 2020

Pipes - percent

Date: / / Page no:

- PercentPipe is used to display data in Percent. i.e 0.51 is displayed as 51 Percent

Syntax

:4.3-4

number_expression | percent [: {minIntegerDigits}. {minFractionDigits}. {maxFractionDigits}

Where minIntegerDigits, minFractionDigits and MaxFraction Digits have default values 1, 0. and 3 respectively.

Pipes - decimal.

DecimalPipe is used to display data in decimal with custom formatting.

number_expression | number [: {minIntegerDigits}. {minFraction Digits}-{maxFractionDigits}

where default values 1, 0, 3 respectively

Pipes - Currency

- CurrencyPipe displays data with currency format.

Syntax:

number_expression | currency [: currencyCode [: display [: {minInteg Digits}. {min minFractionDigits}-{maxFractionDigits}

- {{ mynumber | currency : 'INR' }}
{{ mynumber | currency : 'INR' : 'code' }}

Pipes - JsonPipe

Date: / / Page no:

JsonPipe Converts a value into JSON String.

Syntax

expression | Json

Example

{{ countries | Json }}.

Custom Pipes

Pipes - JsonPipe.

- To create a pipe in Angular, you have to apply the @Pipe decorator to Class, which we can import from the core Angular library.
- The @Pipe decorator allows you to define the pipe name that you'll use within template expression.

Syntax

```
import { Pipe, PipeTransform } from '@angular/core';
  @Pipe({ name: 'Pipename' })

export class PipeClass implements PipeTransform {
    transform(Parameters): returntype { }
}
```

Note: Transform() method will decide the input types, the number of arguments and its types and output type of our custom pipe.

Example: ts file.

```
employees : any = [ {
  code: '001', name: 'Ajeet', gender: 'male', salary: 55003; } ];
```

ng g pipe mypipe --flat

Date: / / Page no:

mypipe.pipe.ts

```
transform(value : string , gender : string) : string {
    if(gender.tolowercase() == "male")
        return "mr" + value;
    else
        return "miss" + value;
}
```

App.component.html

```
<table  border = "1" >
    <tr *ngFor = "let  employee of employees" >
        <td> {{ employee.code }} </td>
        <td> {{ employee.name | mypipe : employee.gender }} </td>
        <td> {{ employee.gender }}  </td>
        <td> {{ employee.salary }} </td>
    </tr>
</table>
```

A barrel is the name for an index.ts file. A barrel is a way to roll-up exports from several modules into a single module.

Date: / / Page no:

bootstrap:

- bootstrap is an open-source repository that provides a couple of native Angular widgets which are built using Bootstrap 4 CSS. As you know Bootstrap is one of the most Popular CSS Frameworks which is used to build stylish and modern applications. which has HTML, CSS and Javas-cript libraries. In order to use Bootstrap frameworks in the Angular app, earlier we would have had to use bootstrap CSS as well as Javascript files, but "ng-bootstrap" gives us the best approach to use without any JS file in our Angular applications

- No 3rd party libraries required.

- Since "bootstrap" itself a module that encapsulates all of the Bootstrap functionality So, there is no need to use any other 3rd party library into our Angular app. As a result, there is no dependency even on JQuery or on Bootstrap's Javascript Library. Only dependencies are

  - Angular 4 and above
  - Bootstrap 4.0.0 (CSS file only, Bootstrap.JS and Bootstrap.min.JS are not required).

npm install bootstrap --save

Template DRiven forms.

Date: / / Page no:

Template Driven Forms Features.

- Easy to use
- Suitable for simple scenarios and fails for complex scenarios.
- Similar to Angular 1.0
- Two way data binding (using [(NgModel)] syntax)
- Minimal component code.
- Automatic track of the form and its data
- Unit testing is another chalange

Ng Form

- It is the directive which helps to create the control groups inside form directive. it is attached to the <Form> element in HTML and supplements form tag with some additional features. Some interesting things we can say about view are that whenever we use the directive in the application view, we need to assign some selector with it and the form is the said selector in our case. The next thing that we need to consider is the ngModel attribute that we have assigned to each HTML control. Let's see what this attribute does.

NgModel.

- When we add NgModel Directive to the control, all the inputs are registered in the Ngform.
- It creates the instance of the formControl class from Domain model and assigns it to the form control element. This control keeps track of the user information and the state and validation status of the Form control.

Next important thing to consider is that when we use NgModel with the form tag or most importantly with the Ngform, we make use of the name property of HTML control, when we look at the control is assigned a name property and we have added the ngmodel attribute

Date: / / Page no:

- Two main functionalities offered by NgForm and NgModel are the permission to retrieving all the values of the control associated with the form and then retrieving the overall state of controls in the form.
- To expose ngform in the application, we have used the following code snippet

```
<form #ngForm = 'ngform' (ngSubmit) = "Register(regForm)">
```

- So this, we are exporting the ngform value in the local variable "regform". Now, the question arises, whether or not we need to use this local variable. Well the answer is no. we are exporting ngform in the local variable just to use some of the properties of the form and these properties are

1. regForm.value : it gives the object containing all the values of the field in the form.
2. regform.valid : This gives us the value indicating if the form is valid or not if it is valid is true else value is false.
3. regform.touched : it returns true or false when one of the field in the form is entered and touched.

In the above case, we have noticed that the form tag has no action method or attribute specified there. So how do we post the data in the component ?

- Let's have a look at the (ngSubmit) = "Register(regform)". Here, we are using the Event binding concept and we are binding which will call the Register method in the component. instead of the submit event of the form, we are using ngSubmit which will send the actual HTTP ~~request~~ request instead of just submitting the form.

Validation.

Date: / / Page no:

The following are the Classes which will be attached whenever the state is changed.

* ng-touched: Controls have been visited.
* ng-untouched: Controls have not been visited.
* ng-dirty: Control value has been Changed.
* ng-Pristine: Controls value have not been Changed
* ng-valid: Control values are valid
* ng-invalid: Control values are invalid.

* All these are return boolean values.

Template Driven Forms with validation

* Here is how

```
input.ng-invalid.ng-touched {
        border-color: red;
}
```